> SECURITY RESEARCH // 2025

HACKING ANDROID

FROM THE INSIDE OUT

A practical look at how Android apps get attacked

> WHO_AM_I.log

JACKSON MAFRA

Mobile Engineer @ Umain

// Android specialist

// Mobile security & pen testing

FIND ME

github.com/jacksonfdam

linkedin.com/in/jacksonmafra

medium.com/@jacksonmafra

@jacksonfdam

// A practical look at how Android apps get attacked — and how to defend them.

> EASTER_EGG // MAY_04.exe

THESE AREN'T THE DROIDS YOU'RE LOOKING FOR

— They all have android:exported='true'

TERMINAL

```
$ adb shell am start \
  -n galaxy.far.away/.R2D2Activity

Starting: Intent { cmp=.../.R2D2Activity }
✓ Exported Activity — no auth required
✓ Death Star plans: EXTRACTED
✓ Login screen bypassed completely

⚠  The Force is android:exported='true'
$ _
```

> 02 / WHY_ANDROID.exe

70% of the World Runs Android

3.5B

ACTIVE DEVICES

- Most targeted mobile OS by attackers
- Apps handle banking, health, identity, payments
- Security is almost always an afterthought

> 03 / ARCHITECTURE.sys

The Architecture Nobody Talks About

- YOUR APP
- JAVA API FRAMEWORK
- NATIVE LIBS + RUNTIME
- HAL — HARDWARE ABSTRACTION
- LINUX KERNEL

// Security decisions happen at every layer.
// And so do mistakes.

> 04 / SANDBOX_MODEL.init

Every App Lives in Its Own Box

App A
UID: 10042

App B
UID: 10043

App C
UID: 10044

// IPC BRIDGE

Controlled inter-process communication only.
No direct access.

// Solid model — but only when devs
// use it correctly.

> 05 / APK_ANATOMY.zip

An APK Is Just a ZIP File

- **AndroidManifest.xml** ← attack surface map
- **classes.dex** compiled app code
- **res/** resources, layouts, strings
- **lib/** native libraries
- **META-INF/** signing certificate

```
$ unzip app.apk -d ./extracted/
$ cat extracted/AndroidManifest.xml
```

> 06 / TOOLS_OF_TRADE.sh

Tools of the Trade

ADB

Android Debug Bridge

Talk directly to any Android device. Install APKs, pull files, shell access.

BURP

Burp Suite

Intercept and manipulate network traffic, including HTTPS.

FRIDA

Frida

Hook into running apps in real time. Inject JS, bypass checks.

MobSF

Mobile Security Framework

Automated static + dynamic analysis. Quick attack surface scan.

> 07 / REVERSE_ENGINEERING.class

Reading Code That Was Never Meant to Be Read

- Apktool → Smali (assembly-level)
- JADX → readable Java
- Hardcoded secrets, API keys, hidden endpoints

```
// compiled .dex (before)
sget-object v0, Lcom/example;
invoke-virtual {v0}, Lcom/
example;→getApiKey()Ljava
/lang/String;
```

▼ JADX decompiles to Java

```
// readable Java (after)
String apiKey = "sk_live_8f2a...";
String endpoint = "/internal/admin";
String secret = "hunter2";
```

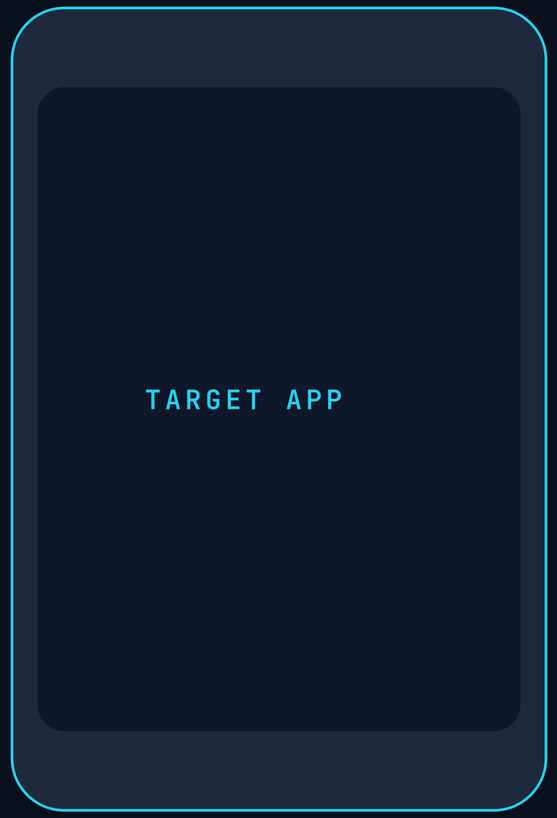
> 08 / ATTACK_SURFACE.map

Where Things Go Wrong

Exported Activities

Exported Services

Broadcast Receivers



Deep Links

WebViews

Insecure Storage

SQL Injection

// Every arrow is a potential entry point. Let's walk through each one.

> 09 / EXPORTED_COMPONENTS.vuln

Leaving the Door Unlocked

```
// AndroidManifest.xml
<activity
  android:name=".AdminActivity"
  android:exported="true" />
```

exported = any app can trigger this component

No access controls = open door for attackers

```
// EXPLOIT
Login screen exported=false
Dashboard exported=true

→ Skip login, launch dashboard
directly.
```

> 10 / DEEP_LINKS.attack

Malicious Links That Open Your App

```
// crafted attack link
myapp://transfer
  ?amount=10000
  &to=attacker_acct
```

→

Apps register custom URL schemes
`myapp://action?param=value`

Parameters not validated → attacker controls the action

Triggered from browser, email, SMS, or another app

> 11 / WEBVIEW_EXPLOIT.js

A Browser Inside Your App

What could go wrong?

```
// Danger zone
webView.addJavascriptInterface(
  new AndroidBridge(), "Android"
);
// JS can now call native code!
```

WebView
JS runs here

→
JS Bridge

Native Code
Files, sensors, apps

If attacker controls the URL loaded → injects JS → calls into native Android code.

> 12 / INSECURE_STORAGE.db

Sensitive Data Left in Plain Sight

STORAGE TYPE	WHAT'S STORED	RISK
Shared Prefs	Tokens, flags, user prefs	Plain XML on disk
SQLite DB	User data, messages, logs	Readable via ADB
Cache Files	Downloads, session data	Forgotten & left behind

// FIX: Encrypt sensitive data.
// Don't store what you don't need.

> 13 / SQL_INJECTION.db

Yes, SQLi Exists in Mobile Apps Too

User Input
unsanitized

→

Content Provider
exposes DB query

```
// Classic injection, same as web
SELECT * FROM users
WHERE name = '' OR '1'='1'
→ Returns ALL rows. All users exposed.
```

```
// FIX: parameterized queries
db.query("users", null,
  "name = ?", new String[]
{input}, ...);
```

Hooking Into a Running App

Inject JS agent into any live process

Hook any function — intercept args, change return values

Bypass SSL pinning, root detection, auth logic

NO SOURCE CODE NEEDED

```
// Frida agent — bypass root detection
Java.perform(function() {
  var Main = Java.use('com.app.Main');
  Main.isRooted.implementation = function() {
    return false; // ← override!
  };
});
```

ATTACH (non-rooted — embedded Gadget)

```
# 1. launch — screen freezes (Gadget waiting)
$ adb shell am start \
    -n com.hackdroid.demo/.MainActivity
# 2. forward Gadget TCP port
$ adb forward tcp:27042 tcp:27042
# 3. attach Frida over TCP
$ frida -H 127.0.0.1:27042 Gadget \
    -l bypass_root_detection.js
[HackDroid] ✓ Root detection bypassed
```

What This Means for Builders and Defenders

// ATTACKER MINDSET

- Your APK is public and readable
- Your manifest declares the attack surface
- Local storage is trivially accessible on rooted devices
- Runtime hooks bypass your logic completely

// DEFENDER ACTION

- Audit your manifest — lock exported components
- Encrypt sensitive data, store only what's needed
- Validate every input — deep links, parameters, WebViews
- **Test your own app the way an attacker would**

> SESSION_END.log

Android Security Is a Mindset

Understand the architecture

Know your attack surface

Think like an attacker

Build like a defender

// Questions?